

Optimizing Quantum Compilation via High-Level Quantum Instructions

Rosa, Marchi, Duzzioni, de Santiago (UFSC, 2025)

Outline

Motivation

Foundations

Theorem 1: Controlled gate reduction

Theorem 2: Approximate decomposition

Theorem 3: Safe uncomputation

V-chain example

Closing

The central thesis

Main argument

Current quantum programming is *circuit-centric* — akin to writing assembly. High-level instructions give the compiler **semantic information** that unlocks optimizations otherwise invisible at the gate level.

- Each transformation the compiler applies is an opportunity for optimization
- This paper proves three such transformations are safe under the `with around` instruction

State of the art

Programming language layer

- **Qiskit, Cirq, Q#** — circuit-centric platforms, explicit gate placement
- **Ket** — qubit-centric circuits
- **Silq** (Bichsel et al., 2020) — safe uncomputation as a language primitive
- **with around instruction** — introduced in Rosa et al. (2024)

Decomposition techniques

- **V-chain** (Barenco et al., 1995)
- **Approx Toffoli** (Maslov, 2016) 3-CNOT decomposition with controlled error
- **Linear-depth controlled gates** (Da Silva & Park, 2022)

Foundational ideas

- **Uncomputation** (Bennett, 1973) — reversible computing, garbage collection
- **Quantum compilation pipeline** — (Javadi-Abhari et al., 2024)
- **Qubit mapping problem** (Li et al., 2019; Zhu et al., 2020)

What this paper adds

- **Theorem 2:** proves **with around** enables safe approximate substitution
- **Theorem 3:** proves precise conditions for automatic uncomputation
- Some **compile-time guarantees**

The compilation pipeline

1. **Gate decomposition** — multi-qubit gates become sequences of 1- and 2-qubit gates
2. **Qubit mapping** — logical qubits assigned to physical qubits respecting hardware connectivity
3. **Native gate translation** — gates lowered to the hardware's ISA (e.g., {RZ, SX, X, CNOT})

The cost metric

On NISQ hardware, **CNOT count** dominates error: CNOTs have $\sim 10\times$ the error rate of single-qubit gates. Reducing CNOTs reduces total circuit error and shortens execution time.

The with around instruction

Ket's high-level instruction for the pattern $A^\dagger BA$:

```
with around(A, qubits):  
    # body B applied here  
# A^dagger applied automatically
```

- Equivalent to: apply A , do B , apply $A^\dagger$
- The instruction **tells the compiler** the segment has the $A^\dagger BA$ structure
- This enables the optimizations proved in Theorems 1, 2, and 3

Controlled gates: $C^n U$

A controlled gate $C^n U$ has the projector-sum form:

$$C^n U = \sum_{k=0}^{2^n-2} |k\rangle\langle k| \otimes I + |2^n-1\rangle\langle 2^n-1| \otimes U$$

Theorem 1 — statement

Theorem 1 (Controlled gate reduction)

For any unitaries A and B :

$$C^n(A^\dagger BA) = A^\dagger \cdot C^n B \cdot A$$

- When outer controls wrap an `with` around segment, the controls only need to fire on B
- A and $A^\dagger$ pass through **unchanged** — no need to build their controlled versions
- Direct CNOT savings whenever A has an expensive controlled decomposition

Approximate decompositions

Definition

An approximate decomposition $\tilde{U}$ differs from U only by a diagonal phase:

$$\tilde{U} = D \cdot U$$

where D is a diagonal unitary (phases on the diagonal, identity structure otherwise).

Example — The approximate Toffoli:

- Exact Toffoli: 6 CNOTs in decomposition
- Approximate Toffoli: 3 CNOTs, but $\tilde{T}|010\rangle = -|010\rangle$
- In isolation, the -1 phase corrupts the computation
- Inside the right structure, the phase **cancels** — safe to use

Theorem 2

Given a circuit segment $P^\dagger D \cdot C^n U \cdot P$ acting on:

- P-register: the qubits P acts on
- U-register: the qubits U acts on

Hypotheses:

- (H1) P is a permutation gate on the P-register
- (H2) D is diagonal on the P-register (phases depend only on P-register basis states)
- (H3) $C^n U$ uses the P-register as control, U-register as target (U arbitrary)
- (H4) $\tilde{P} = D_p P$ where D_p is diagonal on the P-register

Conclusion

Replacing P with $\tilde{P}$ everywhere produces the **identical final unitary**.

Theorem 3 — statement

Theorem 3 (Safe uncomputation)

Given the circuit segment $A^\dagger BA$ where:

- Auxiliary register α is initialized to $|0 \dots 0\rangle$
- A is a permutation gate
- B uses α only as a control of further operations

Then α returns to $|0 \dots 0\rangle$ separable from the rest.

Why this matters:

- Auxiliary qubits can be **safely freed** after the block
- No risk of residual entanglement corrupting later operations
- Enables dynamic memory management for quantum programs

What is novel about uncomputation here?

Prior art:

- with around already inserts $A^\dagger$ automatically
- Uncomputation as a concept dates to Bennett (1970s)
- Silq (2020) formalized uncomputation as a language feature

What Theorem 3 contributes:

- Inserting $A^\dagger$ does **not** automatically return auxiliary qubits to $|0\rangle$ — it depends on B 's structure
- Theorem 3 gives precise structural conditions under which uncomputation is **provably** safe
- The compiler can statically verify these conditions
- Enables the `@using_aux` decorator for dynamic auxiliary allocation

The V-chain decomposition

Problem: implement a multi-controlled NOT gate $C^n X$ on current hardware (only 1- and 2-qubit gates available).

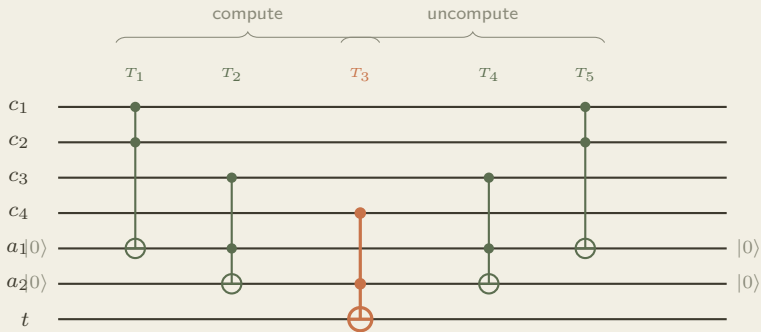
V-chain strategy:

- Use auxiliary qubits to accumulate the AND of controls incrementally
- Each Toffoli combines the running AND with the next control
- Final auxiliary qubit controls a CNOT on the target
- Run all Toffolis in reverse to clean up auxiliaries

Cost trade

Ancilla-free exact decomposition: $O(n^2)$ CNOTs. V-chain with $n - 2$ ancilla: $O(n)$ CNOTs.

V-chain for 4 controls — the circuit



T_1, T_2, T_4, T_5 (moss) get the approximate substitution. T_3 (rust) stays exact — it is the base case, no matched adjoint inside a with around.

The two implementations

With with around:

```
@using_aux(a=lambda c: int(len(c)>2))
def v_chain(c, t, a):
    if len(c) <= 2:
        ctrl(c, X)(t)
    else:
        with around(ctrl(c[:2], X), a):
            v_chain(a + c[2:], t)
```

Without:

```
def v_chain_x(c, t, a):
    if len(c) <= 2:
        ctrl(c, X)(t)
    else:
        ctrl(c[:2], X)(a[0])
        v_chain_x(
            a[0]+c[2:], t, a[1:]
        )
        ctrl(c[:2], X)(a[0])
```

Same logical operation, different compilation

Left: compiler sees the $A^\dagger B A$ structure and applies Theorems 2 and 3. Right: compiler sees three independent Toffolis and cannot optimize.

CNOT count comparison

Version	Toffolis	CNOTs each	Total
Original (4-control)	$T_1, T_2, T_3, T_2^\dagger, T_1^\dagger$	6, 6, 6, 6, 6	30
Optimized (4-control)	$\tilde{T}_1, \tilde{T}_2, T_3, \tilde{T}_2^\dagger, \tilde{T}_1^\dagger$	3, 3, 6, 3, 3	18
Original (6-control, paper)	8 Toffolis + base	—	54
Optimized (6-control, paper)	8 approx + base	—	30

Asymptotic behavior

As the number of controls grows, the CNOT reduction approaches the 50% theoretical maximum.

Simulation results

n	Optimized	Explicit	Reduction
3	14	18	22.2%
4	18	30	40.0%
5	24	42	42.9%
6	30	54	44.4%
7	36	66	45.5%
8	42	78	46.2%
9	48	90	46.7%
10	54	102	47.1%
11	60	114	47.4%
12	66	126	47.6%
13	72	138	47.8%
14	78	150	48.0%
15	84	162	48.1%
16	90	174	48.3%
17	96	186	48.4%
18	102	198	48.5%
19	108	210	48.6%

What we reproduced

- Ran Ket 0.9.3 locally on macOS
- Same algorithm as Figure 4
- CNOT counts read from logical gate metadata

Headline result

- $n = 6$ controls: **30 vs 54 CNOTs**
- Asymptotic reduction approaches **50%**

What it shows

- The compiler does the work, transparently

What is novel

Novel contributions:

- **Theorem 2:** proves approximate decomposition is safe inside with around
- **Theorem 3:** proves precise structural conditions for automatic safe uncomputation
- Lifts these from programmer obligations to **compile-time guarantees**
- Enables @using_aux dynamic auxiliary qubit allocation

Further questions

Open directions:

- Dirty auxiliary qubits (initialized to unknown state)
- Broader static analysis to identify safe substitution patterns
- Integration with the qubit mapping stage of compilation
- Further high-level instructions exposing additional semantic structure

Questions?

Optimizing Quantum Compilation via
High-Level Quantum Instructions

Rosa, Marchi, Duzzioni, de Santiago (UFSC, 2025)
arXiv:2510.24323